

Apunte de Clase: Microarquitectura - Pipeline Superescalar en RISC

Introducción

El pipeline escalar (visto en el apunte anterior) permite ejecutar una instrucción por ciclo de reloj en el mejor de los casos. Sin embargo, el paralelismo a nivel de instrucción (ILP) presente en muchos programas es mayor. Para aprovecharlo, se necesita emitir múltiples instrucciones por ciclo, lo que da origen a los pipelines superescalares.

Un procesador superescalar tiene múltiples pipelines paralelos y puede ejecutar varias instrucciones simultáneamente (por ejemplo, 2, 3, 4 o más instrucciones por ciclo). Esto requiere:

- Múltiples unidades funcionales (ALUs, unidades de carga/almacenamiento, etc.).
- Lógica de control compleja para manejar dependencias.
- Técnicas avanzadas de predicción de saltos y ejecución fuera de orden.

1. Del Escalar al Superescalar

1.1 Limitaciones del pipeline escalar

En un pipeline escalar:

- CPI (Ciclos Por Instrucción) ideal = 1.
- El CPI real = CPI ideal + stalls estructurales + stalls por datos + stalls por control.
- Aunque usemos forwarding y predicción simple de saltos, a lo sumo emitimos una instrucción por ciclo.

1.2 Idea del superescalar

Para superar la barrera de 1 instrucción/ciclo, se construyen múltiples pipelines en paralelo. En cada ciclo, la etapa de fetch trae un grupo de instrucciones (por ejemplo, 2, 4 u 8) y se distribuyen entre varios pipelines.

Ejemplo: Un procesador superescalar de 2 vías (2-issue) puede ejecutar 2 instrucciones por ciclo si no hay dependencias entre ellas.

2. Dependencias entre Instrucciones

Para poder ejecutar instrucciones en paralelo, es fundamental entender los distintos tipos de dependencias. Las dependencias reales (verdaderas) no pueden eliminarse; las falsas (antidependencias y dependencias de salida) pueden eliminarse mediante renombrado de registros.

2.1 Dependencias Verdaderas (RAW - Read After Write)

- Definición: La instrucción j lee un registro que la instrucción i escribe.
- También llamada: Dependencia de flujo de datos verdadera.
- Ejemplo:

assembly

I1: ADD R1, R2, R3 ; escribe R1

I2: SUB R4, R1, R5 ; lee R1 (depende de I1)

- Consecuencia: I2 no puede ejecutarse hasta que I1 produzca su resultado.
- Preservación: El orden debe mantenerse para que el programa sea correcto.

2.2 Antidependencia (WAR - Write After Read)

- Definición: La instrucción i lee un registro que la instrucción j escribe después.
- También llamada: Dependencia falsa (no hay flujo de datos real).
- Ejemplo:

assembly

I1: ADD R4, R1, R2 ; lee R4

I2: SUB R4, R3, R5 ; escribe R4 (después de I1)

- Problema: Si se reordenan, I1 podría leer el nuevo valor de R4 (incorrecto).
- Solución: Se puede eliminar mediante renombrado de registros.

2.3 Dependencia de Salida (WAW - Write After Write)

- Definición: Dos instrucciones escriben el mismo registro.
- También llamada: Dependencia falsa de salida.
- Ejemplo:

assembly

I1: ADD R1, R2, R3 ; escribe R1

I2: SUB R1, R4, R5 ; también escribe R1 (después)

- Problema: El valor final debe ser el de la última escritura (I2).
- Solución: Se elimina con renombrado (se le asigna un registro físico distinto a cada escritura).

2.4 Dependencias de Control

- Ocurren con instrucciones de branch (saltos condicionales).
- Una instrucción depende de que un branch sea tomado o no para saber si debe ejecutarse.
- Solución: Predicción de saltos (estática o dinámica).

3. Predicción de Saltos (Branch Prediction)

3.1 Predicción Estática

El compilador o el hardware usan reglas fijas (no aprenden del comportamiento en ejecución).

- Saltos demorados (delayed branch): El compilador coloca una instrucción útil después del branch (siempre se ejecuta, antes de decidir si salta). Usado en MIPS temprano.
- Predicted taken: Siempre predecir que el salto se tomará.
- Predicted not taken: Siempre predecir que el salto no se tomará.

Problema: Mala precisión (~50-70% en promedios, peor en algunos patrones).

3.2 Predicción Dinámica

El hardware aprende del comportamiento pasado de cada branch.

Buffer de Predicción de Saltos (Branch Prediction Buffer)

- Una tabla indexada por parte de la dirección de la instrucción de salto.
- Cada entrada tiene 1 bit que indica si la última vez se tomó (1) o no (0).
- Problema: Un solo bit es impreciso (cambia de opinión demasiado rápido).

Dirección	Bit (1=tomado)
0x34123	1
0x19923	0
0x21339	1

Predictor de 2 bits

- Usa 2 bits por entrada, implementando una máquina de estados de 4 estados (fuerte tomado, débil tomado, débil no tomado, fuerte no tomado).
- Ventaja: Se necesitan dos predicciones erróneas consecutivas para cambiar de dirección. Es mucho más robusto.

Diagrama de estados (implícito en el documento):

- 11 (fuerte tomado) → 10 (débil tomado) → 01 (débil no tomado) → 00 (fuerte no tomado), y viceversa.
- Ejemplo con dos bits: Si un branch se toma 9 de cada 10 veces, el predictor de 2 bits acertará muchas más veces que uno de 1 bit.

4. Planificación Dinámica (Dynamic Scheduling)

4.1 Limitaciones de la planificación estática

- El compilador organiza el código para minimizar stalls, pero no conoce los eventos en tiempo de ejecución (fallos de caché, latencias variables, etc.).
- Los stalls frenan todo el pipeline.

4.2 Idea de la planificación dinámica

El hardware reordena la ejecución de las instrucciones para mantener las unidades funcionales ocupadas, aunque las instrucciones se hayan emitido en orden.

- Ejecución fuera de orden (Out-of-Order Execution, OoO): Las instrucciones comienzan a ejecutarse tan pronto como sus operandos están disponibles, no necesariamente en el orden del programa.
- Finalización en orden (In-Order Commit): Aunque se ejecuten fuera de orden, los resultados se escriben (commit) en el orden original del programa. Esto es esencial para manejar excepciones correctamente.

4.3 Riesgos nuevos introducidos por OoO

- En pipelines escalares con planificación estática, WAR y WAW no se manifiestan porque las instrucciones siempre se ejecutan en orden.
- Con OoO, estos riesgos pueden aparecer y deben gestionarse.

Ejemplo de WAR (en OoO):

assembly

I1: ADD R4, R1, R2 ; lee R4

I2: SUB R4, R3, R5 ; escribe R4

Si I2 se ejecuta antes que I1 (porque sus operandos están listos), I1 leería el valor escrito por I2, lo cual es incorrecto. Se debe evitar.

5. Algoritmo de Tomasulo

5.1 Origen e importancia

- Desarrollado por Robert Tomasulo para la IBM 360 (década de 1960).
- Originalmente para la unidad de punto flotante.
- Sigue siendo la base de los procesadores modernos (Intel Core, AMD Ryzen, ARM Cortex-A) aunque con mejoras.

5.2 Componentes clave

1. Reservation Stations (RS): Pequeñas memorias asociadas a cada unidad funcional (ADD, MUL, etc.). Cada instrucción pendiente tiene su propia RS que:
 - Almacena los operandos (en cuanto están listos).

- Realiza renombrado de registros (elimina WAR y WAW).
 - Espera hasta que todos los operandos estén disponibles para ejecutarse.
2. Register Alias Table (RAT): Tabla que indica, para cada registro lógico (ej: R1), qué Reservation Station (o registro físico) contiene su valor más reciente.
 3. Common Data Bus (CDB): Bus por donde se transmiten los resultados de las unidades funcionales hacia todas las Reservation Stations y el banco de registros.
 4. Reorder Buffer (ROB) (opcional, pero usado en implementaciones modernas): Buffer que mantiene las instrucciones en orden hasta que se confirman (commit).

5.3 Funcionamiento paso a paso (simplificado)

Ejemplo (del documento):

Instrucciones:

ADD R1, R2, R3 ; $R1 = R2 + R3$

SUB R4, R1, R2 ; $R4 = R1 - R2$

DIV R1, R2, R3 ; $R1 = R2 / R3$

Estado inicial:

- Registros aritméticos: $R1=3.14$, $R2=-1$, $R3=2.72$, $R4=0.71$
- RAT vacía (cada registro apunta a sí mismo o a RS).

Proceso:

1. Issue (emisión): Se decodifica ADD y se asigna una Reservation Station (RS1). La RAT se actualiza: $R1 \rightarrow RS1$. Los operandos (R2, R3) ya están listos, se copian a RS1.
2. Issue SUB: Se detecta que R1 está en RS1. No se toma el valor del banco de registros (sería viejo). En su lugar, se anota que el operando vendrá de RS1 (renombrado). Se asigna RS2. RAT: $R4 \rightarrow RS2$.
3. Issue DIV: Se detecta que R1 está en RS1. Se renombra: el operando vendrá de RS1. Se asigna RS3. RAT: $R1 \rightarrow RS3$ (ahora apunta a DIV, no a ADD).

Resultado del renombrado: WAW entre ADD y DIV desapareció (escriben en distintas RS). WAR entre ADD y SUB también desapareció (SUB lee el valor de RS1, no de R1 directamente).

4. Ejecución: Cuando RS1 termina, envía el resultado por CDB. Todas las RS que lo esperaban (RS2, RS3) lo reciben y lo almacenan. Luego RS2 y RS3 pueden ejecutarse.
5. Commit: Los resultados se escriben en los registros reales en el orden del programa.

5.4 Ventajas del método de Tomasulo

- Elimina WAR y WAW mediante renombrado.
- Permite ejecución fuera de orden: cada instrucción se ejecuta cuando sus operandos están listos.
- Tolerancia a latencias variables (ej: fallos de caché, divisiones lentas) – otras instrucciones pueden seguir ejecutándose.
- No requiere que el compilador conozca la microarquitectura (aunque ayuda).
- Base para la especulación por hardware (ejecución predictiva de branches).

6. Ejecución Especulativa

6.1 Motivación

Los riesgos de control (branches) siguen siendo una barrera. Si predecimos mal, perdemos ciclos. La ejecución especulativa permite:

- Predecir un branch y ejecutar instrucciones más allá del branch antes de saber si la predicción fue correcta.
- Si la predicción es correcta → gran ganancia.
- Si es incorrecta → se descartan los resultados (se deshacen) y se penaliza.

6.2 Componentes necesarios

- Predictor de saltos (de 2 bits o más avanzado, como TAGE).
- Reorder Buffer (ROB): Buffer circular que mantiene los resultados en orden de programa hasta que se confirman (commit) o se descartan.
- Estado no comprometido (speculative): Los resultados de instrucciones especulativas no se escriben en los registros hasta que se confirme que el branch era correcto.

6.3 Flujo típico

1. Fetch y predicción de branch.
2. Emisión de instrucciones especulativas.
3. Ejecución (fuera de orden).
4. Commit en orden: Cuando un branch se resuelve:
 - Si fue bien predicho → se confirman las instrucciones especulativas.
 - Si fue mal predicho → se descartan (ROB se vacía) y se redirige el flujo a la dirección correcta.

7. Resumen de Soluciones a las Dependencias

Tipo de dependencia	Nombre	¿Se puede eliminar?	Solución típica
RAW (Read After Write)	Verdadera	No	Forwarding + stalls mínimos; OoO con Tomasulo
WAR (Write After Read)	Falsa (antidependencia)	Sí	Renombrado de registros (Tomasulo)
WAW (Write After Write)	Falsa (dependencia de salida)	Sí	Renombrado de registros (Tomasulo)

Control (branch)	Dependencia de flujo	Parcialmente	Predicción dinámica + ejecución especulativa
Estructural	Recurso compartido	Sí (a veces)	Duplicar recursos; planificación dinámica

El pipeline superescalar con ejecución fuera de orden, renombrado de registros (Tomasulo) y predicción dinámica de saltos es la base de todos los procesadores de alto rendimiento actuales, desde Intel Core hasta Apple M-series y AMD Ryzen.
Ecuación fundamental del rendimiento (vista en el documento):

$$\text{CPI_real} = \text{CPI_ideal} + \text{stalls_estructurales} + \text{stalls_datos} + \text{stalls_control}$$

El objetivo del superescalar es:

- Minimizar CPI_ideal (emitiendo múltiples instrucciones por ciclo).
- Reducir los stalls mediante las técnicas vistas.

Limitación práctica: El ILP disponible en programas típicos (especialmente los no científicos) es limitado. La ley de Amdahl sigue vigente: incluso con un superescalar de 8 vías, muchas aplicaciones no alcanzan una aceleración de 8× debido a dependencias y branches.